Certified Collapse of Tseitin Formulas via XOR Detection and Sparse Elimination

Witnora-Core Artifact

v1.0-public-preprint package

Abstract

We present Witnora-Core, a certified collapse-path engine that transforms CNF formulas through deterministic, replayable steps into tractable classes when possible, reconstructs witnesses, and exports externally verifiable DRAT proofs for supported UNSAT cases. We demonstrate the system on official CNFgen Tseitin formulas. Witnora detects hidden XOR structure, applies a deterministic sparse/min-fill elimination order to control proof size, and generates DRAT proofs verified by `drat-trim`. In the recorded v0.9.8 diagnostic, Witnora externally verifies seven official CNFgen Tseitin rows, including `tseitin_20 c=2`, where the pre-XOR v0.8.4 baseline reported no collapse on the corresponding Tseitin diagnostic. The method is deterministic and reproducible from the released source and artifact package. We explicitly limit the claim to exact-pattern XOR extraction on the recorded Tseitin matrix and do not claim parity support, industrial benchmark performance, or competitive SAT solving.

1 Introduction

Witnora-Core studies a narrow systems question: can a deterministic, replayable collapse engine turn selected CNF formulas into certified SAT or UNSAT results without using an external SAT solver for solving? Earlier Witnora releases established a collapse-path framework, certificate bundles, an independent raw checker, public result categories, benchmark artifacts, and DRAT export for supported UNSAT paths.

The v0.8.4 official CNFgen diagnostic showed a clear gap: the Tseitin rows in the recorded matrix remained `NO_COLLAPSE_FOUND`. The v0.9 series added exact CNF XOR extraction and a bounded XOR-to-DRAT proof bridge. The final v0.9.8 diagnostic records the before/after result: the selected official Tseitin rows move from no-collapse to externally checked DRAT proofs under the recorded configuration.

This note is a short systems report. It makes row-level claims about a reproducible artifact, not broad claims about SAT solving.

2 Background

CNF SAT. A CNF formula is a conjunction of clauses, where each clause is a disjunction of literals. A SAT certificate is an assignment satisfying the original formula. An UNSAT certificate requires evidence that no assignment satisfies the original formula. Modern SAT solvers are commonly based on CDCL and related proof-logging infrastructure [7, 1, 8, 3, 2].

Collapse paths. Witnora-Core represents computation as a collapse path: a deterministic sequence of simplification, decomposition, restricted solving, reconstruction, and verification steps. A collapse path is accepted only when it can be replayed and checked against the original input.

Tseitin formulas. Tseitin formulas encode parity constraints over graph-like structure into CNF [11]. They are useful here because their algebraic structure is explicit enough for an exact XOR detector to identify, while still being invisible to the earlier treewidth-focused route under the recorded diagnostic. The formulas in this artifact were generated with CNFgen 0.9.5 [6].

XOR and affine structure. An XOR equation over GF(2) has the form

$$x_1 \oplus x_2 \oplus \cdots \oplus x_k = b.$$

Witnora-Core v0.9.8 detects only exact full-CNF encodings of such equations up to the configured arity cap. It does not perform heuristic XOR mining. XOR reasoning and Gaussian elimination are established techniques in SAT systems, including cryptographic SAT work [10].

DRAT proof checking. DRAT proof checking validates clausal proof artifacts [12, 5]. Witnora-Core treats native XOR inconsistency as an internal trace and counts original-CNF UNSAT as externally certified only when `proof.drat` is accepted by real `drat-trim` against `original.dimacs`.

3 Witnora-Core Architecture

Deterministic collapse grammar. The core collapse grammar includes unit propagation, pure literal elimination, subsumption, blocked clause elimination, component decomposition, bounded treewidth solving, 2-SAT solving, and exact XOR reasoning for explicitly supported inputs. Bounded-width and decomposition-based SAT approaches provide context for this structural route [4, 9].

Certificate bundles. Witnora certificate bundles store `original.dimacs`, normalized formulas, collapse paths, reconstruction traces, result metadata, proof traces, DRAT status, and hashes. Timing is not part of the certificate hash.

Independent checker. The independent raw checker parses raw DIMACS, recomputes supported transformations from raw clauses, checks SAT assignments against the original formula, replays native XOR traces, and verifies that public categories match the proof metadata. It does not replace `drat-trim` for DRAT proof validation.

Public result categories. Public reports separate logical result, discovery route, and certification level. The categories used in this note are:

- `UNSAT_DRAT_EXPORTED_VERIFIED`: `proof.drat` was verified by real `drat-trim` against `original.dimacs`.
- `UNSAT_XOR_NATIVE_ONLY`: a native XOR contradiction exists, but no external original-CNF DRAT proof is claimed.
- `NO_COLLAPSE_FOUND`: Witnora did not find a collapse under the recorded grammar and parameters.

4 XOR Collapse Layer

Exact CNF XOR extraction. For variables $V = \{x_1, \ldots, x_k\}$ and right-hand side b, the full CNF encoding of an XOR equation contains one clause for each assignment to V whose parity is not b. Witnora detects such encodings only when the exact source clauses are present. Source clauses are retained in the CNF.

Sidecar distinction. Sidecar XOR equations define a combined problem: CNF plus XOR sidecar. They are not claimed to be implied by the original CNF. This note concerns detected-CNF XOR equations whose source clauses are already in `original.dimacs`.

Gaussian elimination. Gaussian elimination over GF(2) finds native XOR contradictions when the detected equations are inconsistent. Free variables are handled deterministically in SAT cases.

Deterministic sparse/min-fill elimination. The sparse/min-fill order is a deterministic heuristic used to reduce peak row width during XOR elimination. The order and row-operation trace are recorded in the certificate and independently replayed. It is not claimed to be optimal.

Table 1: v0.8.4 vs. v0.9.8 Tseitin results.

Instance and c	v0.8.4 category	v0.9.8 category	XOR eqs.	max arity	peak width	clauses	bytes	trim sec.	profile
tseitin_10 c=2	NO_COLLAPSE_FOUND	UNSAT_DRAT_EXPORTED_VERIFIED	10	4	6	575	12381	0.073599	default
tseitin_10 c=3	NO_COLLAPSE_FOUND	UNSAT_DRAT_EXPORTED_VERIFIED	10	4	6	575	12381	0.072742	default
tseitin_10 c=4	NO_COLLAPSE_FOUND	UNSAT_DRAT_EXPORTED_VERIFIED	10	4	6	575	12381	0.072092	default
tseitin_15 c=2	NO_COLLAPSE_FOUND	UNSAT_DRAT_EXPORTED_VERIFIED	15	4	8	3999	124485	0.090604	default
tseitin_15 c=3	NO_COLLAPSE_FOUND	UNSAT_DRAT_EXPORTED_VERIFIED	15	4	8	3999	124485	0.092360	default
tseitin_15 c=4	NO_COLLAPSE_FOUND	UNSAT_DRAT_EXPORTED_VERIFIED	15	4	8	3999	124485	0.093661	default
tseitin_20 c=2	NO_COLLAPSE_FOUND	UNSAT_DRAT_EXPORTED_VERIFIED	20	4	10	7775	277861	0.121430	medium-v098

5 Certification Model

SAT assignment verification. SAT results are certified by reconstructing an assignment and checking it against the original CNF and any active XOR equations.

Native XOR trace replay. A native XOR trace records GF(2) row operations. The independent checker accepts the native contradiction only when replay reaches $0 = 1$.

DRAT export. For detected-CNF XOR contradictions, Witnora emits a clausal proof guided by the recorded XOR trace. The proof base is the original CNF, not a sidecar or modified formula.

`drat-trim` verification. The certification authority for externally checked UNSAT rows is real `drat-trim`. Witnora reports an externally verified UNSAT category only when `drat-trim` accepts `proof.drat` against `original.dimacs`.

6 Experimental Setup

- Witnora tag: `v0.9.8`
- Witnora commit: `27e1876b5d815e3f3ba21392395b0e0e9dbcd025`
- CNF generator: official CNFgen `0.9.5`
- CNFgen path used: `C:\Users\maham\AppData\Local\hermes\hermes-agent\venv\Scripts\cnfgen.exe`
- Proof checker: `C:\Users\maham\AppData\Local\Temp\witnora-drat-trim-real\drat-trim\drat-trim.exe`
- Proof checker SHA256: `74D6D8FA2330C8D633B4E380CFFAD22517546C94963D4D2C381FEDF16641E1A8`
- The local `drat-trim` binary used for the recorded run prints a usage banner for `-h` but does not expose a version string through that command.
- Source archive SHA256: `26557CD73DC45DF60EF5E056699AFC34E3402645DE60EFE050C5CE2ADADC267F`
- Diagnostic artifact SHA256: `4702299E28CDB4E3B16B0CD1CBFE9BA96018E625C0E8864A1C0476C2AEC3FA2C`
- Clean extraction tests: 188 passing

The v0.8.4 baseline reported the official Tseitin diagnostic rows as `NO_COLLAPSE_FOUND`. The v0.9.8 matrix uses `--detect-xor`, `--detect-xor-max-k 6`, `--xor-elim-order sparse-min-fill`, `--xor-drat`, and real `drat-trim`.

Table 2: Gaussian vs. deterministic sparse/min-fill on `tseitin_15`.

Order	peak width	public category	proof clauses	proof bytes
Gaussian	14	`UNSAT_XOR_NATIVE_ONLY`	0	0
Sparse/min-fill	8	`UNSAT_DRAT_EXPORTED_VERIFIED`	3999	124485

Table 3: Row-width cap sensitivity for `tseitin_20 c=2`.

Profile	cap	peak width	category	proof clauses	bytes
default	8	10	cap skipped	0	0
medium-v098	10	10	`UNSAT_DRAT_EXPORTED_VERIFIED`	7775	277861

7 Results

Parameter sensitivity and the row-width cap. The row-width cap limits the maximum permitted XOR row width during proof export. It is a safety guard against proof explosion, not a theorem claim. `tseitin_10` and `tseitin_15` verified under cap 8. The `tseitin_20 c=2` row exceeded cap 8 with peak row width 10, so the default profile was reported as a native-only cap skip. Under the documented medium profile with cap 10, the same algorithm exported a DRAT proof verified by `drat-trim`. The proof remained small in this diagnostic: 7775 proof clauses, 277861 bytes, and approximately 0.121 seconds of `drat-trim` runtime.

For context, the earlier focused `tseitin_10 c=2` XOR-DRAT diagnostic reported peak row width 8, 10143 proof clauses, 388389 proof bytes, and approximately 0.140 seconds of `drat-trim` runtime. The paper claim uses the final v0.9.8 matrix artifact values shown above.

8 Theorem and Proof Notes

Exact XOR pattern soundness. If a set of CNF clauses exactly equals the forbidden-assignment CNF encoding of an XOR equation E, then every assignment satisfying those clauses satisfies E. Adding E as an implied constraint while retaining the source clauses is sound for SAT witnesses and native XOR reasoning.

XOR elimination trace soundness. A recorded GF(2) row-operation trace is sound if every row is obtained by XOR-combining previous or source rows and the final row is $0 = 1$. This establishes native XOR inconsistency of the detected equation system.

DRAT certification boundary. Native XOR inconsistency is not, by itself, an external proof of original CNF UNSAT. Witnora counts UNSAT as externally certified only when `proof.drat` is verified by real `drat-trim` against `original.dimacs`.

Sparse/min-fill determinism. Sparse/min-fill is a deterministic heuristic order recorded in the certificate and independently replayed. It is not claimed to be optimal.

9 Limitations and Scope

- Exact XOR patterns only.
- Detection cap $k \leq 6$ in this diagnostic.
- Recorded Tseitin diagnostic rows only.
- No parity result is claimed.
- No industrial benchmark result is claimed.

Table 4: Proof size and runtime summary from the final v0.9.8 matrix.

Instance	peak width	proof clauses	proof bytes	trim sec.
`tseitin_10 c=2`	6	575	12381	0.073599
`tseitin_10 c=3`	6	575	12381	0.072742
`tseitin_10 c=4`	6	575	12381	0.072092
`tseitin_15 c=2`	8	3999	124485	0.090604
`tseitin_15 c=3`	8	3999	124485	0.092360
`tseitin_15 c=4`	8	3999	124485	0.093661
`tseitin_20 c=2`	10	7775	277861	0.121430

- No SAT solver competitiveness result is claimed.
- No full DRAT/RAT/LRAT/FRAT support is claimed.
- No general SAT solving claim is made.
- The row-width cap is a configurable proof-export safety parameter, not a theorem boundary.
- Sparse/min-fill is deterministic and useful in this diagnostic, but it is not claimed to be optimal.

10 Related Work and Positioning

Witnora-Core is positioned among CDCL SAT solving and proof logging, SAT preprocessing, certified solving, XOR-aware reasoning, bounded-width and decomposition-based SAT approaches, and crafted benchmark generation.

CDCL SAT solvers combine conflict-driven search with preprocessing and can emit proof logs for UNSAT instances [7, 1, 8, 3, 2]. Witnora is not presented as a competitive SAT solver. It is a certified collapse-path prototype that emphasizes deterministic structural routes, replayable certificates, and external proof verification for supported cases.

SAT preprocessing and structural simplification reduce formulas through transformations such as subsumption, blocked clause elimination, variable elimination, and decomposition. Witnora records these transformations as collapse paths and assigns explicit public categories to the result.

XOR reasoning and Gaussian elimination have a long history in SAT systems, especially for formulas with algebraic structure [10]. Witnora does not claim novelty for XOR reasoning itself and does not implement broad or heuristic XOR mining. The v0.9.8 route is narrower: it accepts exact full-CNF XOR encodings and retains the source clauses.

DRAT proof checking and related proof formats provide external UNSAT certification [12, 5]. Witnora uses real `drat-trim` as the authority for rows counted as externally verified. Native XOR traces are replayed independently, but they are not treated as replacements for externally checked clausal proofs.

Bounded-width and decomposition-based methods motivate Witnora's collapse framework [4, 9], while CNFgen provides crafted benchmark families such as Tseitin formulas for controlled diagnostics [6]. The reported result is therefore best read as a reproducible systems artifact: exact-pattern XOR detection plus deterministic sparse/min-fill trace recording plus externally checked DRAT proof export for the recorded official CNFgen Tseitin rows.

11 Reproducibility

The reproducibility package records the source archive, diagnostic artifact, commands, expected category counts, proof locations, `drat-trim` logs, and independent checker output locations. The key diagnostic command is:

```
$env:WITNORA_DRAT_TRIM='...\drat-trim.exe'
python -m witnora_core.xor_drat_diagnostic `
  --matrix-preset full-tseitin-v098 `
  --detect-xor-max-k 6 `
```

```
--xor-drat `
--elim-orders sparse-min-fill `
--profiles default medium-v098 `
--drat-trim $env:WITNORA_DRAT_TRIM `
--cnfgen-path 'C:\Users\maham\AppData\Local\hermes\hermes-agent\venv\Scripts\cnfgen.exe' `
--out-root outputs\v0.9.8-full-tseitin-minfill-diagnostic `
--zip witnora-core-v0.9.8-full-tseitin-minfill-diagnostic.zip
```

Expected best-matrix public category counts are:

- `UNSAT_DRAT_EXPORTED_VERIFIED`: 7
- `UNSAT_XOR_NATIVE_ONLY`: 0
- `NO_COLLAPSE_FOUND`: 0
- `TIMEOUT`: 0
- `ERROR`: 0

12 Conclusion

The v0.9.8 artifact establishes a bounded certified systems result: selected official CNFgen Tseitin diagnostic rows that were previously no-collapse are externally DRAT-verified after exact CNF XOR extraction and deterministic sparse/min-fill proof export. The result is intentionally narrow, but it is reproducible, externally checked, and separated from native-only or skipped cases by public result categories.

A Artifact Appendix

Item	Value
Source archive	`witnora-core-v0.9.9-paper-rev1.zip`
Source archive SHA256	`9217E14D16DCF87A8A9F14BEC7DE24C52EB3F6C2C7CF4DDF88415D73D0F3B49D`
Frozen source tag	`v0.9.9-paper-rev1`
Frozen source commit	`6039db406451f60143586060b5773db6bfa6d216`
Diagnostic artifact	`witnora-core-v0.9.8-full-tseitin-minfill-diagnostic.zip`
Diagnostic artifact SHA256	`4702299E28CDB4E3B16B0CD1CBFE9BA96018E625C0E8864A1C0476C2AEC3FA2C`
drat-trim executable SHA256	`74D6D8FA2330C8D633B4E380CFFAD22517546C94963D4D2C381FEDF16641E1A8`
Test command	`python -m unittest discover -s tests -v`
Expected tests	188 passing with real `drat-trim` enabled

References

[1] Roberto J. Bayardo and Robert C. Schrag. Using CSP look-back techniques to solve real-world SAT instances. In *Proceedings of the Fourteenth National Conference on Artificial Intelligence*, pages 203–208, 1997.

[2] Armin Biere, Marijn Heule, Hans van Maaren, and Toby Walsh, editors. *Handbook of Satisfiability*, volume 185 of *Frontiers in Artificial Intelligence and Applications*. IOS Press, 2009.

[3] Niklas Een and Niklas Sorensson. An extensible SAT-solver. In *Theory and Applications of Satisfiability Testing - SAT 2003*, volume 2919 of *Lecture Notes in Computer Science*, pages 502–518. Springer, 2004.

[4] Georg Gottlob, Nicola Leone, and Francesco Scarcello. Hypertree decompositions and tractable queries. *Journal of Computer and System Sciences*, 64(3):579–627, 2002.

[5] Marijn J. H. Heule. The DRAT format and DRAT-trim checker, 2016.

[6] Massimo Lauria. CNFgen: Combinatorial benchmarks for SAT solvers. Software, version 0.9.5. Used to generate the official Tseitin diagnostic formulas in the recorded artifact.

[7] Joao P. Marques-Silva and Karem A. Sakallah. GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521, 1999.

[8] Matthew W. Moskewicz, Conor F. Madigan, Ying Zhao, Lintao Zhang, and Sharad Malik. Chaff: Engineering an efficient SAT solver. In *Proceedings of the 38th Design Automation Conference*, pages 530–535. ACM, 2001.

[9] Marko Samer and Stefan Szeider. Constraint satisfaction with bounded treewidth revisited. *Journal of Computer and System Sciences*, 76(2):103–114, 2010.

[10] Mate Soos, Karsten Nohl, and Claude Castelluccia. Extending SAT solvers to cryptographic problems. In *Theory and Applications of Satisfiability Testing - SAT 2009*, volume 5584 of *Lecture Notes in Computer Science*, pages 244–257. Springer, 2009.

[11] G. S. Tseitin. On the complexity of derivation in propositional calculus. In A. O. Slisenko, editor, *Studies in Constructive Mathematics and Mathematical Logic, Part II*, pages 115–125. Steklov Mathematical Institute, 1970. Translated from Russian; original appeared in Zapiski Nauchnykh Seminarov LOMI 8, 1968.

[12] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In *Theory and Applications of Satisfiability Testing - SAT 2014*, volume 8561 of *Lecture Notes in Computer Science*, pages 422–429. Springer, 2014.